

Methane & CO2 Tracker

Relatório da Fase 3 — API REST, Notificação de Alertas e Dashboard

Escopo desta fase: expor os dados armazenados (leituras e alertas) via uma API REST autenticada, notificar automaticamente por e-mail e webhook quando um alerta é aberto, e entregar a primeira versão do dashboard visual consumindo essa API.

Decisões de arquitetura desta fase

- API REST em FastAPI (*app/api/*) com três routers (readings, alerts, devices), todos protegidos por dependência **require_api_key** — header *X-API-Key* validado contra *settings.api_key_secret*. Sem sessão/cookie: pensado para consumo por dashboard e integrações externas.
- Paginação uniforme via *app/api/pagination.py* (offset/limit, resposta envelopada em **Page[T]** com total) e filtros por *device_id*, *gas_type* e janela de tempo (*start/end*) no endpoint de leituras — mesmos filtros aplicáveis a alertas.
- Notificação de alerta (*app/alerts/notify.py*) em dois canais independentes: e-mail via SMTP (Mailpit em dev, container novo no docker-compose) e webhook HTTP opcional. **notify_alert()** isola cada canal em try/except próprio — uma falha de notificação nunca derruba a ingestão — e marca *Alert.notified_at* se pelo menos um canal entregou.
- Dashboard em React 18 + TypeScript + Vite, consumindo a API via *services/api.ts*. Autenticação simples por API key guardada no browser (*ApiKeyGate.tsx*) — sem fluxo de login completo, adequado ao estágio atual do produto. Visualização de leituras recentes (*ReadingsChart.tsx*, recharts) e eventos de alerta (*AlertsList.tsx*).
- Nenhuma migração de schema nova foi necessária nesta fase — a coluna *notified_at* já existia desde a 0001_initial_schema.

Arquivos entregues

Arquivo	Papel
backend/app/api/main.py	App FastAPI, registro dos routers
backend/app/api/deps.py	Autenticação por API key + sessão de DB
backend/app/api/pagination.py	Helper de paginação offset/limit
backend/app/api/schemas.py	Schemas Pydantic de resposta (Page, ReadingOut, AlertOut...)
backend/app/api/routers/readings.py	GET /readings — filtros por sensor/gás/período
backend/app/api/routers/alerts.py	GET /alerts — filtros por sensor/gás/status
backend/app/api/routers/devices.py	GET /devices
backend/app/alerts/notify.py	Notificação por e-mail (SMTP) e webhook
docker-compose.yml	Serviço mailpit (SMTP dev) adicionado
frontend/src/pages/Dashboard.tsx	Página principal do dashboard
frontend/src/components/ReadingsChart.tsx	Gráfico de leituras recentes (recharts)
frontend/src/components/AlertsList.tsx	Lista de eventos de alerta
frontend/src/components/ApiKeyGate.tsx	Gate de autenticação por API key

frontend/src/services/api.ts	Cliente HTTP da API REST
backend/tests/test_api.py	Teste standalone — executado, OK
backend/tests/test_alert_notifications.py	Teste standalone — executado, OK

Status de verificação

- Ambiente subido via *docker compose up -d timescaledb mosquitto mailpit* — os três serviços confirmados saudáveis.
- test_api.py: **executado com sucesso** — cobre autenticação obrigatória (401 sem/incorrecta API key), paginação e filtros por gás, período e status.
- test_alert_notifications.py: **executado com sucesso** — alerta disparado notifica por e-mail (capturado no Mailpit) e por webhook (receptor local de teste), e *notified_at* é marcado no banco.
- Dashboard: build de produção (*tsc -b && vite build*) executado com sucesso, sem erros de tipo.

Não incluído nesta fase (próximos passos)

- Detecção de anomalia (Isolation Forest) — módulo app/anomaly ainda vazio.
- Geração de relatório PDF ANP/EPA do produto — módulo app/reports ainda vazio.
- Fluxo de login completo no dashboard (hoje: API key única compartilhada) e code-splitting do bundle React (>500kB no build atual).

Gerado automaticamente por backend/scripts/generate_phase3_report.py